DynamicViT: Making Vision Transformer faster through layer skipping

Amanuel Mersha
School of Information Technology and Engineering
Addis Ababa Institute of Technology
Addis Ababa, Ethiopia
amanuel.negash@aau.edu.et

Sammy Assefa
U.S. Bank
1 Federal St.
Boston, MA, USA
samuel.assefa@usbank.com

Abstract

Recent deep learning breakthroughs in language and vision tasks can be mainly attributed to large-scale transformers. Unfortunately, the massive size and high compute requirement of these models have limited their use in resource-constrained environments. Dynamic neural networks present a unique opportunity to reduce the amount of compute requirement as these models enable dynamically adjusting the computational path given an input. We propose a layer-skipping dynamic vision transformer (ViT) network that skips layers for each sample based on decisions given by a reinforcement learning agent. Extensive experiments on CIFAR-10 and CIFAR-100 showed that this dynamic ViT model gains an average of 40% throughput increase in the inference phase when evaluated on different batch sizes ranging from 1 to 1024.

1 Introduction

Transformers have had a major transformational force behind the recent success of deep learning [5, 10, 2, 12]. These models, at their core, employ a multi-head self-attention mechanism that enables them to explicitly define relationships between inputs in terms of attention score. This mechanism has been shown to perform significantly better than previous methods in NLP tasks [4, 1, 9]. Furthermore, the self-attention building block increases the model's explainability, helping researchers to understand neural networks by probing the attention heads. Later, Dosovitskiy et al. [6] showed that transformers could also be applied to vision tasks with little to no modification and perform on par with CNN-based models. This unique ability of transformers, generalizing to different kinds of tasks (vision, language, multi-modal learning), made them more ubiquitous in the literature.

Parallel to the Transformer's success, Reinforcement learning (RL) has recently seen impressive success. It has been used to solve various problems such as robot control [11], AutoML [21] and games [14]. At the core, these systems benefit from RL because it enables them to learn complex non-differentiable reward functions. Deep learning itself, as a typical problem environment, can benefit from such a learning system too. Dynamic neural networks modify their inner workings (parameters, layers, connectivity) based on an input sample [18, 8, 17, 20]. By employing RL in these dynamic networks, it is possible to introduce learned sparsification which results in efficient deeper models.

In this work, we present the DynamicViT that uses an RL agent that determines the computational path for a given input. Given the *CLS* token of an input, an RL agent learns whether to skip a layer or not. Thus, each input may have different routes in the forward propagation. The agent is trained using *REINFORCE with baseline* algorithm with a reward function that increases accuracy and decreases the number of activated layers per input. The resulting model is faster when compared to a vanilla vision transformer counterpart.

Vision Transformers: Theory and Applications Workshop (VTTA@NeurIPS 2022).

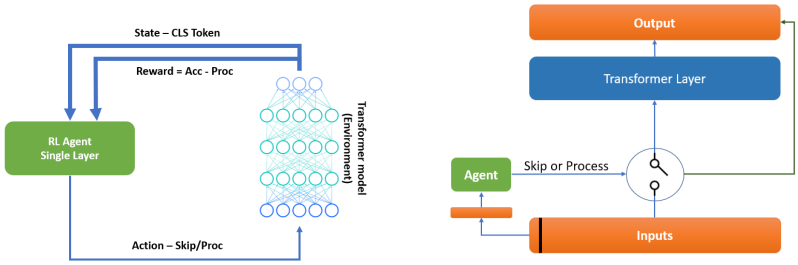

Figure 1: RL Agent Schematics: The agent views the transformer network as a typical RL environment (Left). The sensory input is the *CLS* Token (Right). The actions are skipping a layer or not.

2 Related Works

Dehghani et al. [3] introduced the Universal Transformer (UT) architecture which applied the concept of recurrence, not over the input symbols, but over the depth-wise revision of the symbols. The technique only uses a single layer, which is repeatedly executed over the inputs. This enables the model to determine the depth length, and thus incorporate dynamic halting that is applied per-symbol level. Hence, some of the symbols in a given input may not halt with other symbols, in which case, the successive depth is just a copy of the halted symbols.

Elbayad et al. [7] improved the Universal Transformer by increasing the model capacity through incorporating additional layers. Their model, the Adaptive Depth Transformer, introduced several ways to estimate the depth and exit strategies. The token-specific depth determines the depth for every token in a block. For the sequence-specific depth estimation, an oracle function, given the average of the tokens in a block, predicts the depth.

Wang et al. [18] proposed SkipNet which is partly an inspiration to our work. SkipNet employed gating modules that decide whether to skip residual blocks or not. The gating functions are implemented in three ways: FFGate-I and FFGate-II are standard CNN-based networks where FFGate-II has a higher capacity. RNNGate on the other hand uses Recurrent Neural Network (RNN) in order to share parameters across multiple blocks. The work also introduced a reinforcement learning agent (Hybrid-RL) that mitigates the non-differentiable function of executing the gates.

In this work-in-progress, we propose a dynamic vision transformer network (*DynamicViT*) that skips layers based on policy decisions given by a reinforcement learning agent. The resulting model, not only is faster, but also more accurate than a typical fixed network.

3 Models

The architecture has two parts: a typical image classification model trained in a supervised manner and an RL agent. The classification model is a vanilla transformer network whereas, the agent is a collection of linear layers residing at each transformer layer. These sub-agents learn decision functions that enable the transformer to skip the corresponding layer or not. The parameters are trained in parallel but, gradients are updated separately as stability issues arise if trained in a multi-task fashion.

Conventions: For the equations below, x_j represents inputs to the jth transformer layer T_j (MSA + FFN) [16]. a_j represent activations of T_j. In a typical transformer model, $a_j = x_{j+1}$ as the activations pass without any modification. However, the formulations presented here apply some kind of function on the a_j resulting in different x_{j+1}. x_0 is the image patch, $x_1 = a_0$ is the encoding of the patches into the transformer's *hidden* size. a_1 is the output of the first transformer layer T_1 and $x_2 = a_1$.

Simoulin and Crabbé [15] hypothesized that the *CLS* token aggregates the representation of the entire input sequence, which is also supported by the fact that this token is used for classification purposes in the final layer of ViTs [6]. Hence, one can safely assume that based on this *CLS* token, an agent can issue a decision on what process should be applied to the corresponding sequence. For brevity,

c_j represents the *CLS* token in x_j. π_j is the policy function learned by the agent at layer j and $\pi_j(c_j) \in \{0, 1\}$. The agent is described in subsection 3.1.

At each layer, the agent issues decisions for each sample on whether to skip layer T_j or not based on c_j. Those samples x_j^T with $\pi_j(c_j) = 1$ are extracted from the batch and pass through the T_j. The remaining, x_j^I, will be cloned to become a_j^I (identity function). After the first group passes through the transformer layer T_j, the activations a_j^T will be merged with the a_j^I. The merging is simply placing each activation in its original batch index. The following equations show how the skipping mechanism is implemented in a given layer.

$$
\begin{aligned}
s_j &= \pi_j(c_j) \\
x_j^T &= x_j[s_j == 1] \\
x_j^I &= x_j[s_j == 0] \\
a_j^T &= T_j(x_j^T) \\
a_j^I &= x_j^I \\
a_j &= merge(a_j^T, a_j^I) \\
x_{j+1} &= a_j
\end{aligned}
$$

3.1 Skip Policy Agent

Each of the transformer's layers T_j has an accompanying agent made of a linear layer that predicts whether T_j should be applied to the incoming inputs x_j or not. Figure 1 depicts this mechanism. These linear layers are trained using policy gradient with the following RL specifications:

- **Action Space**: The actions are *pass an input through a layer* (**1**) or *skip that layer* (**0**).
- **State Space**: The input is the *CLS* token, c_j, which is continuous spaces.
- **Reward**: The reward function is designed to increase the accuracy of the model while minimizing the number of activated layers per input. Hence, overall, it is defined as:

  $$Reward = Accuracy - Process_Ratio$$

  For each input i in the training examples, the accuracy $A^i \in \{0, 1\}$ is set to whether the input i was correctly classified or not. The processing ratio P^i is the fraction of the activated layers when processing the input i, computed as:

  $$P^i = \frac{\sum_{j=0}^{L} \pi_j(c_j^i)}{L}$$

  where $\pi_j(c_j^i) \in \{0, 1\}$ and L is the total number of layers. The per-input reward (or the return) for the last layer's sub-agent is then:

  $$G_L^i = R_L^i = A^i - P^i$$

  As the task is a finite horizon RL problem, the reward can be propagated down to the first time step, which is the first layer. However, at each layer, other than the last layer, the immediate reward is 0. Hence, the return for each time step is simply a discounted future return given by:

  $$G_j^i = \gamma \times G_{j+1}^i$$

  The final reward matrix G will have a shape of $(batch_size \times num_layers)$ as it is computed per-input and per layer. Whitening is then applied to this matrix along the num_layers dimension. The baseline network's loss for the jth layer is a mean squared error, computed as:

  $$BL_j = \frac{1}{n} \sum_{i=1}^{n} (G_j^i - b(c_j^i))^2$$

  where n is the number of inputs in a batch and b is the baseline function that computes the learned value of a state. Finally, the policy network's loss for the jth layer is computed as:

  $$PL_j = \frac{1}{n} \sum_{i=1}^{n} (G_j^i - b(c_j^i)) log \pi_j(c_j^i)$$

Model	Layers	CIFAR-10	CIFAR-100	Ave. Throughput
Vanilla ViT	7	89.24	65.43	3581
Dynamic ViT	7	89.34	67.26	4948 (38%)
Vanilla ViT	10	89.37	66.51	2575
Dynamic ViT	10	89.96	66.89	3737 (45%)
Vanilla ViT	16	88.88	67.36	1251
Dynamic ViT	16	**90.17**	**67.62**	3072 (145%)

Table 1: Performance of the models on CIFAR datasets. The models were run for 150 epochs (300 for DynamicViT). In both tasks, in all layer amounts, DynamicViT outperforms by a small margin. *Ave. Throughput* is the average number of images processed per microsecond on two GPUs (RTX Titan, Google ColabPro) and 10 batch sizes, ranging from 1 to 1024.

REINFORCE with baseline[19] algorithm was used to train the agent. The policy network is a single-layer feed-forward network whereas the baseline network is a two-layer feed-forward neural network. In the policy network case, increasing the number of layers didn't improve upon the single-layer FFN. Hence, to avoid extra computation, the single-layer FFN was used. In the baseline's network case, however, a two-layer network was better than a single one. We further observed that sharing the parameters across the layers (through LSTM or the same FFN) severely hurts both the skip ratio and accuracy of the model. Precisely, the agent issues almost the same decision in all the layers despite being fed different *CLS* tokens. Without the whitening, the model either results in very poor performance or halts due to numerical stability issues. Furthermore, when the accuracy and processing ratio become equal, the reward becomes zero. Such reward creates numerical instability. Hence, we added $\epsilon = -10^{-5}$ on the reward so that such cases can be avoided. A positive epsilon value showed a small drop in the accuracy of the model.

4 Experiment and Result

To evaluate our method, we first implemented a smaller scale vanilla ViT model with $mlp_hidden = hidden = 384$. We then trained 7, 10, and 16-layer models on CIFAR-10 and CIFAR-100. The full list of hyperparameters is specified in table 3. The skip policy agent was then added to this base model. The performances of the models presented here are all lower than state-of-the-art as recent works employ significantly larger models that are pre-trained on Imagenet[13].

As the batch size increases from 256 to 1568, all models perform better. However, in the dynamic transformer case, we found that it is consistently and significantly sub-optimal when compared to the base model for the same batch size. Precisely, to achieve a comparable level of performance, it needed twice the batch size of the base model. Investigating this, we discovered that, due to the skipping mechanism, the model processes about half of the given batch in a layer, which explains why it requires twice the batch size to compete with the baseline model. Hence, to make a fair comparison, the batch size was doubled from 784 to 1568 and the number of epochs is multiplied by two to achieve a similar level of training steps.

4.1 Performance

Table 1 shows the performance of the models. The models were first trained for 150 (*300 for DynamicViT*) epochs. To determine if more epochs have an effect, we trained the 7 and 10-layer models for 300 epochs (600 for DynamicViT). The Vanilla ViT showed better performance on the CIFAR-10 as figure 4. Overall, however, it shows that the method doesn't put pressure on the accuracy-speed trade-off. This is because it effectively utilizes the bottom layers and only employs the later ones if a sample requires extra processing. Figure 6 shows that the model applies the earlier layers for all samples in the batch. However, later layers are activated for a small fraction of the batch.

4.2 Inference Speed

We compared the throughput of the Dynamic ViT in batch sizes from 1 to 1024 on Google ColabPro and RTX Titan GPUs. It is computed as the number of images processed in a microsecond. Except

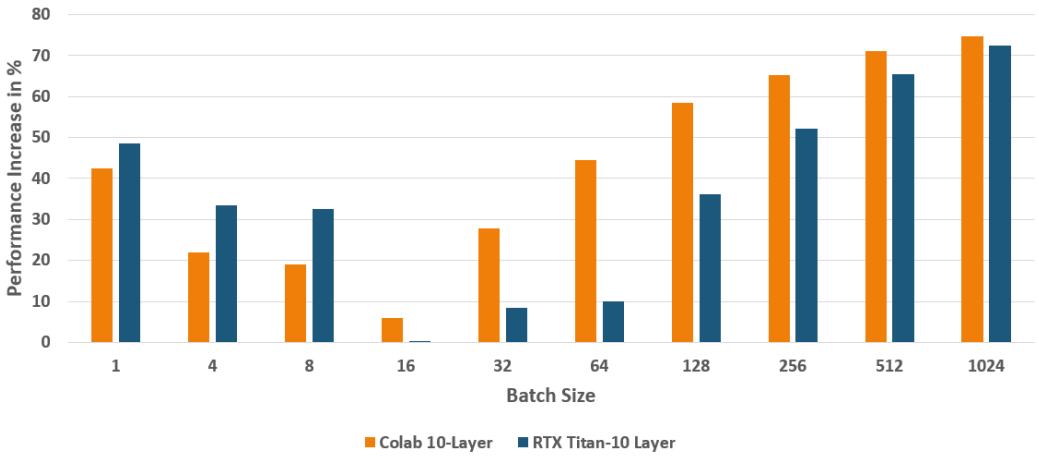

Figure 2: Throughput improvement (in %) of 10-layer DynamicViT over the vanilla ViT in batch sizes from 1 - 1024

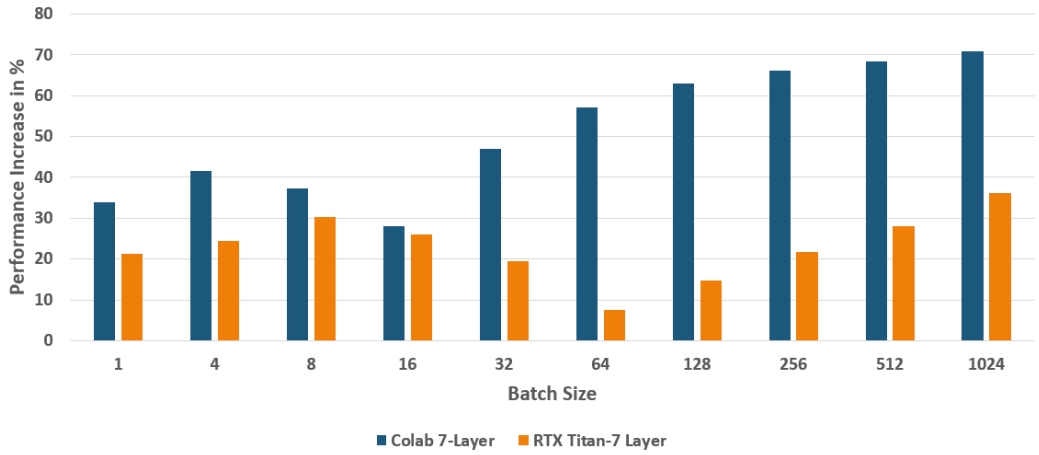

Figure 3: Throughput improvement (in %) of 7-layer DynamicViT over the vanilla ViT in batch sizes from 1 - 1024

for the 16 batch sizes, there is a large throughput increase in all batch sizes. Averaging over all batch sizes, Google ColabPro gains about 43% throughput increase while RTX Titan gains about 35%. Figure 2 and 3 show the throughput percentage increase of the 10 and 7 layer ViTs compared to fixed networks. ColabPro uses a single thread and hence, it benefits highly (up to 266%) in a bigger (16 layers) model. Figure 7 shows the speed increase in the 16-layer DynamicViT case. Table 2 shows the average percentage increase of the models on the two GPUs. Overall, the method increases throughput in all of our settings. With the exception of the 16 batch size, both GPUs show a significant throughput increase.

The increase in speed is attributed to the later layers. Earlier layers process all of the inputs and later layers are selective. Figure 6 shows the fraction of the inputs in a batch that are being processed on each layer of DynamicViT-7 and DynamicViT-10. Layers 1 to 5 process all inputs. The rest of the layers process much smaller fractions of the batch inputs.

5 Conclusion

In this paper, we presented a prototype, layer skipping dynamic vision transformer called *DynamicViT*. It employs an RL agent to determine if a sample should pass through a layer or skip that layer. The agent is a collection of single-layer feed-forward neural networks that act at each transformer layer

independently. It is trained using *REINFORCE with baseline* algorithm. The resulting model learns to skip a layer per sample, which makes it compute efficient in the inference phase. Averaging over batch sizes ranging from 1 to 1024 over two GPU types, it offers a 40% throughput increase. Online systems such as self-driving cars will greatly benefit from such speed increases. It also translates into a 40% deployment cost reduction, resulting in lower carbon emissions. Scale is important in determining the true performance of deep models and at the moment, we are working on standard benchmarks.

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. J. Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *ArXiv*, abs/2005.14165, 2020.

[2] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek B Rao, Parker Barnes, Yi Tay, Noam M. Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Benton C. Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier García, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Díaz, Orhan Firat, Michele Catasta, Jason Wei, Kathleen S. Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *ArXiv*, abs/2204.02311, 2022.

[3] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. *ArXiv*, abs/1807.03819, 2019.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *ArXiv*, abs/1810.04805, 2019.

[6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *ArXiv*, abs/2010.11929, 2021.

[7] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. *ArXiv*, abs/1910.10073, 2020.

[8] Alex Graves. Adaptive computation time for recurrent neural networks. *ArXiv*, abs/1603.08983, 2016.

[9] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *ArXiv*, abs/1907.11692, 2019.

[10] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9992–10002, 2021.

[11] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *ArXiv*, abs/1709.10087, 2018.

[12] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley D. Edwards, Nicolas Manfred Otto Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. A generalist agent. *ArXiv*, abs/2205.06175, 2022.

[13] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3):211–252, 2015.

[14] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, L. Sifre, Dharshan Kumaran, Thore Graepel, Timothy P. Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *ArXiv*, abs/1712.01815, 2017.

[15] Antoine Simoulin and Benoit Crabbé. How many layers and why? an analysis of the model depth in transformers. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: Student Research Workshop*, pages 221–228, 2021.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[17] Andreas Veit and Serge J. Belongie. Convolutional networks with adaptive inference graphs. *International Journal of Computer Vision*, 128:730–741, 2019.

[18] Xin Wang, Fisher Yu, Zi-Yi Dou, and Joseph Gonzalez. Skipnet: Learning dynamic routing in convolutional networks. *ArXiv*, abs/1711.09485, 2018.

[19] Lex Weaver and Nigel Tao. The optimal reward baseline for gradient-based reinforcement learning. *ArXiv*, abs/1301.2315, 2001.

[20] Brandon Yang, Gabriel Bender, Quoc V. Le, and Jiquan Ngiam. Condconv: Conditionally parameterized convolutions for efficient inference. In *NeurIPS*, 2019.

[21] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. *ArXiv*, abs/1611.01578, 2017.

Model	ColabPro	RTX Titan
DynamicViT 7-layer	51%	23%
DynamicViT 10-layer	43%	35%
DynamicViT 16-layer	266%	46%

Table 2: Speed gain of the DynamicViT models on CIFAR-100 compared to the vanilla ViT.

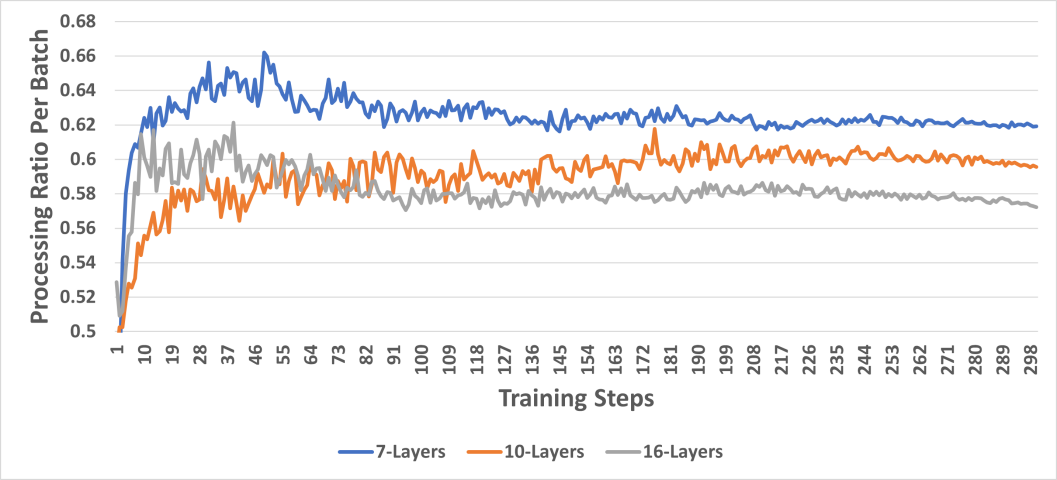

Figure 4: Processing fraction per batch of the models over the training steps: Blue (ViT-7), Orange (ViT-10), and Gray (ViT-16)

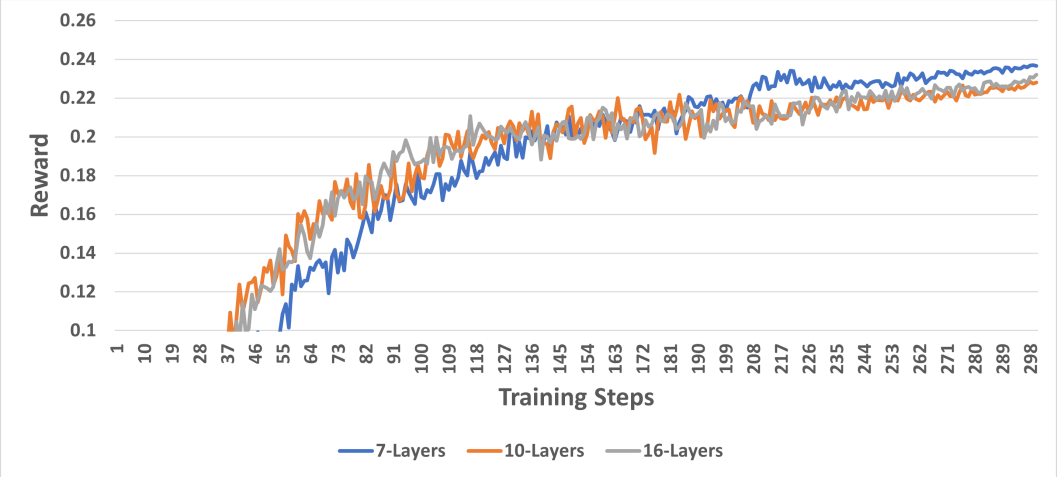

Figure 5: Rewards over training steps: Blue (ViT-7), Orange (ViT-10), and Gray (ViT-16)

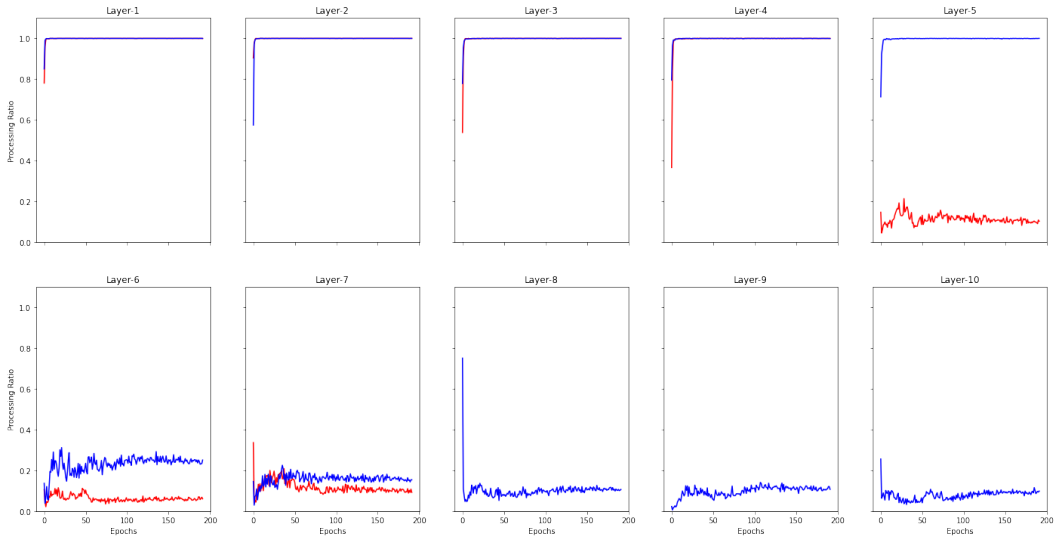

Figure 6: The processing ratio at each layer of ViT-7 (red) and Vit-10 (blue). DynamicViT-7 uses all the first 4 layers. However, starting at 5th layer, it becomes selective by processing less than 20% of the incoming batch. Similarly, DynamicViT-10 utilizes 5 layers and the rest is used selectively.

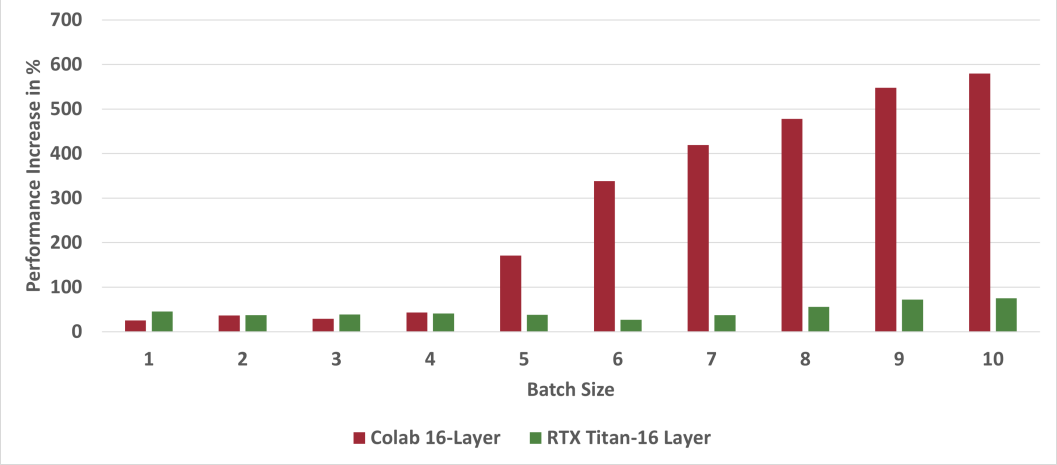

Figure 7: Throughput improvement (in %) of 16-layer DynamicViT over the vanilla ViT in batch sizes from 1 - 1024

Hyperparam	**Value**
Embedding size	384
MLP Hidden	384
Num of layers	7, 10
batch size	768
dropout	0.2
lr	0.001
label smoothing	yes
autoaugment	yes
warm-up epoch	5
precision	16 bit
num of head	12
weight-decay	5e-5

Table 3: Base model hyperparamaters

Model	Layers	CIFAR-10	CIFAR-100	Ave. Throughput
Vanilla ViT	7	89.24	65.46	3581
Dynamic ViT	7	89.35	67.48	4948 (38%)
Vanilla ViT	10	89.28	66.61	2575
Dynamic ViT	10	90.03	**67.66**	3737 (45%)
Vanilla ViT (300 epoch)	7	91.08	67.37	3581
Dynamic ViT (600 epoch)	7	90.16	68.32	4948 (38%)
Vanilla ViT (300 epoch)	10	**91.14**	67.77	2575
Dynamic ViT (600 epoch)	10	90.48	**69.17**	3737 (45%)

Table 4: Performance of the models on CIFAR datasets. The models were run for 300 epochs (600 for DynamicViT). In the 150-epoch case, DynamicViT outperformed the vanilla ViT in all settings. In the 300-epoch case, however, the Vanilla ViT showed better performance on CIFAR-10.